

PREMIER UNIVERSITY, CHITTAGONG

Department of Computer Science & Engineering



PROJECT REPORT

Course Name : Big Data Analytics Laboratory
Course Code : CSE 4346
Name of Report : Student Lifestyle, Academic Performance and
Mental Health Analysis Using Hadoop
MapReduce and Apache Pig
Date of Submission : 15.09.2026

Submitted By,	
Name	: Robiul Hassan
ID	: 0222220005101043
Semester	: 8 th
Batch	: 42 nd
Session	: Spring - 26
Section : A	

<u>Submitted To :</u> MD Tamim Hossain Lecturer, Department of Computer Science & Engineering Premier University	Remarks
---	----------------

Problem Statement

The increasing amount of student-related data makes it difficult to efficiently analyze the relationships between students' academic performance, lifestyle habits, stress levels, sleep duration, study hours, and mental health. Traditional data processing methods may become inefficient when handling large datasets.

This project aims to use Hadoop MapReduce and Apache Pig to process and analyze a dataset containing 100,000 student records. The analysis focuses on identifying academic and lifestyle patterns, comparing different student groups, and extracting meaningful information such as average CGPA, stress levels, sleep duration, study hours, depression status, and top-performing students.

The project demonstrates how big data processing technologies can be used to efficiently process large-scale student data and generate useful analytical results.

Dataset Description

Attribute	Description	Data Type
Student_ID	Unique identification number of each student	Integer
Age	Age of the student	Integer
Gender	Gender of the student	Categorical
Department	Academic department of the student	Categorical
CGPA	Cumulative Grade Point Average	Float
Sleep_Duration	Average daily sleep duration in hours	Float
Study_Hours	Average daily study hours	Float
Social_Media_Hours	Average daily social media usage in hours	Float
Physical_Activity	Physical activity level/duration	Integer
Stress_Level	Student's stress level	Integer
Depression	Indicates whether the student has depression	Boolean

The dataset file used during the experiments is `student_lifestyle_100k.csv`. It contains 100,000 records and is approximately 5 MB in size..

Summary of Data Analysis

The following MapReduce programs and their different design patterns are implemented:

1. Filtering
2. Summarization
3. Counting

The following analyses are performed on the dataset using MapReduce:

1. Calculate Average CGPA by Department
2. Calculate Average Stress Level by Gender
3. Calculate Number of Students by Depression Status

The following analyses are performed on the dataset using Apache Pig:

1. Calculate Top 10 Students by CGPA
2. Calculate Average Study Hours by Department
3. Calculate Average Sleep Duration by Depression Status
4. Calculate Top 10 Students by Study Hours

Map Reduce Analysis

1. Average CGPA by Department:

Returns the average CGPA of students for each academic department. This analysis comprises of:

i. Driver Class : Sets the configuration for the Mapper and Reducer classes and defines the input and output paths.

ii. Mapper Class : Emits the department of type Text as the key and CGPA of type DoubleWritable as the value.

iii. Reducer Class : Calculates the average CGPA for each department and emits the department of type Text and average CGPA of type DoubleWritable.

Driver Class

```
public class AverageCGPADriver {  
  
    public static void main(String[] args) throws Exception {  
  
        if (args.length != 2) {  
            System.err.println("Usage: AverageCGPADriver <input> <output>");  
            System.exit(2);  
        }  
  
        Configuration conf = new Configuration();  
  
        Job job = Job.getInstance(conf, "Average CGPA by Department");  
  
        job.setJarByClass(AverageCGPADriver.class);  
  
        job.setMapperClass(AverageCGPAMapper.class);  
        job.setReducerClass(AverageCGPAReducer.class);
```

```

        job.setMapOutputKeyClass(Text.class);
        job.setMapOutputValueClass(DoubleWritable.class);

        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(DoubleWritable.class);

        FileInputFormat.addInputPath(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));

        System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
}

```

Mapper Class

```

public class AverageCGPAMapper extends Mapper<Object, Text, Text, DoubleWritable>
{

    private final Text department = new Text();
    private final DoubleWritable cgpa = new DoubleWritable();

    @Override
    public void map(Object key, Text value, Context context)
        throws IOException, InterruptedException {

        String line = value.toString().trim();

        if (line.isEmpty() || line.startsWith("Student_ID")) {
            return;
        }

        String[] fields = line.split(",");

        if (fields.length < 11) {
            return;
        }

        try {
            department.set(fields[3].trim());
            cgpa.set(Double.parseDouble(fields[4].trim()));

            context.write(department, cgpa);
        } catch (NumberFormatException e) {
            // Skip malformed records
        }
    }
}

```

```

    }
}
}

```

Reducer Class

```

public class AverageCGPAReducer
    extends Reducer<Text, DoubleWritable, Text, DoubleWritable> {

    private final DoubleWritable result = new DoubleWritable();

    @Override
    public void reduce(Text key, Iterable<DoubleWritable> values, Context
context)
        throws IOException, InterruptedException {

        double sum = 0.0;
        long count = 0;

        for (DoubleWritable value : values) {
            sum += value.get();
            count++;
        }

        if (count > 0) {
            result.set(sum / count);
            context.write(key, result);
        }
    }
}

```

Execution

```

2026-09-10 19:46:21,686 INFO mapreduce.Job: Running job: job_1789047296965_0001
2026-09-10 19:46:27,767 INFO mapreduce.Job: Job job_1789047296965_0001 running in uber mode : false
2026-09-10 19:46:27,767 INFO mapreduce.Job: map 0% reduce 0%
2026-09-10 19:46:32,810 INFO mapreduce.Job: map 100% reduce 0%
2026-09-10 19:46:37,837 INFO mapreduce.Job: map 100% reduce 100%
2026-09-10 19:46:37,837 INFO mapreduce.Job: Job job_1789047296965_0001 completed successfully
2026-09-10 19:46:37,915 INFO mapreduce.Job: Counters: 54

```

Output

```

Arts      2.902723272327235
Business  2.89767403314917
Engineering 2.8971351647803876
Medical  2.896051893408144
Science  2.897992127945791

```

2. Average Stress Level by Gender:

Returns the average stress level of students for each gender. This analysis comprises of:

i. Driver Class : Sets the configuration for the Mapper and Reducer classes and defines the input and output paths.

ii. Mapper Class : Emits the gender of type `Text` as the key and stress level of type `DoubleWritable` as the value.

iii. Reducer Class : Calculates the average stress level for each gender and emits the gender of type `Text` and average stress level of type `DoubleWritable`.

Driver

```
public class AverageStressDriver {

    public static void main(String[] args) throws Exception {

        if (args.length != 2) {
            System.err.println("Usage: AverageStressDriver <input> <output>");
            System.exit(2);
        }

        Configuration conf = new Configuration();

        Job job = Job.getInstance(conf, "Average Stress Level by Gender");

        job.setJarByClass(AverageStressDriver.class);

        job.setMapperClass(AverageStressMapper.class);
        job.setReducerClass(AverageStressReducer.class);

        job.setMapOutputKeyClass(Text.class);
        job.setMapOutputValueClass(DoubleWritable.class);

        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(DoubleWritable.class);

        FileInputFormat.addInputPath(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));

        System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
}
```

Mapper

```
public class AverageStressMapper
    extends Mapper<Object, Text, Text, DoubleWritable> {

    private final Text gender = new Text();
    private final DoubleWritable stress = new DoubleWritable();

    @Override
    public void map(Object key, Text value, Context context)
        throws IOException, InterruptedException {

        String line = value.toString().trim();

        if (line.isEmpty() || line.startsWith("Student_ID")) {
            return;
        }

        String[] fields = line.split(",");

        if (fields.length < 11) {
            return;
        }

        try {
            gender.set(fields[2].trim());
            stress.set(Double.parseDouble(fields[9].trim()));

            context.write(gender, stress);
        } catch (NumberFormatException e) {
            // Skip malformed records
        }
    }
}
```

Reducer

```
public class AverageStressReducer
    extends Reducer<Text, DoubleWritable, Text, DoubleWritable> {

    private final DoubleWritable result = new DoubleWritable();

    @Override
    public void reduce(Text key, Iterable<DoubleWritable> values, Context
context)
```

```

        throws IOException, InterruptedException {

    double sum = 0.0;
    long count = 0;

    for (DoubleWritable value : values) {
        sum += value.get();
        count++;
    }

    if (count > 0) {
        result.set(sum / count);
        context.write(key, result);
    }
}
}

```

Execution

```

2026-09-10 20:11:50,076 INFO mapreduce.Job: Running job: job_1789047296965_0002
2026-09-10 20:11:56,160 INFO mapreduce.Job: Job job_1789047296965_0002 running in uber mode : false
2026-09-10 20:11:56,160 INFO mapreduce.Job: map 0% reduce 0%
2026-09-10 20:12:00,226 INFO mapreduce.Job: map 100% reduce 0%
2026-09-10 20:12:05,273 INFO mapreduce.Job: map 100% reduce 100%
2026-09-10 20:12:05,274 INFO mapreduce.Job: Job job_1789047296965_0002 completed successfully
2026-09-10 20:12:05,343 INFO mapreduce.Job: Counters: 54

```

Output

```

Female 4.130012028869286
Male 4.13330007980846

```

3. Number of Students by Depression Status:

Returns the number of students for each depression status (`True` or `False`). This analysis comprises of:

i. Driver Class : Sets the configuration for the Mapper and Reducer classes and defines the input and output paths.

ii. Mapper Class : Emits the depression status of type `Text` as the key and a count of 1 as the value.

iii. Reducer Class : Sums the counts for each depression status and emits the depression status of type `Text` and total number of students of type `IntWritable`.

Driver

```
public class DepressionCountDriver {

    public static void main(String[] args) throws Exception {

        if (args.length != 2) {
            System.err.println("Usage: DepressionCountDriver <input> <output>");
            System.exit(2);
        }

        Configuration conf = new Configuration();

        Job job = Job.getInstance(conf, "Student Count by Depression Status");

        job.setJarByClass(DepressionCountDriver.class);

        job.setMapperClass(DepressionCountMapper.class);
        job.setReducerClass(DepressionCountReducer.class);

        job.setMapOutputKeyClass(Text.class);
        job.setMapOutputValueClass(IntWritable.class);

        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(IntWritable.class);

        FileInputFormat.addInputPath(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));

        System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
}
```

Mapper

```
public class DepressionCountMapper
    extends Mapper<Object, Text, Text, IntWritable> {

    private final Text depression = new Text();
    private static final IntWritable ONE = new IntWritable(1);

    @Override
    public void map(Object key, Text value, Context context)
        throws IOException, InterruptedException {

        String line = value.toString().trim();
```

```

        if (line.isEmpty() || line.startsWith("Student_ID")) {
            return;
        }

        String[] fields = line.split(",");

        if (fields.length < 11) {
            return;
        }

        depression.set(fields[10].trim());

        context.write(depression, ONE);
    }
}

```

Reducer

```

public class DepressionCountReducer
    extends Reducer<Text, IntWritable, Text, IntWritable> {

    private final IntWritable result = new IntWritable();

    @Override
    public void reduce(Text key, Iterable<IntWritable> values, Context context)
        throws IOException, InterruptedException {

        int sum = 0;

        for (IntWritable value : values) {
            sum += value.get();
        }

        result.set(sum);

        context.write(key, result);
    }
}

```

Execution

```

2026-09-10 20:20:50,636 INFO mapreduce.Job: Running job: job_1789047296965_0003
2026-09-10 20:20:55,723 INFO mapreduce.Job: Job job_1789047296965_0003 running in uber mode : false
2026-09-10 20:20:55,723 INFO mapreduce.Job: map 0% reduce 0%
2026-09-10 20:20:59,784 INFO mapreduce.Job: map 100% reduce 0%
2026-09-10 20:21:04,805 INFO mapreduce.Job: map 100% reduce 100%
2026-09-10 20:21:04,806 INFO mapreduce.Job: Job job_1789047296965_0003 completed successfully
2026-09-10 20:21:04,865 INFO mapreduce.Job: Counters: 54

```

Output

```
False 89938
True 10062
```

Pig Analysis

1. Top 10 Students By CGPA: Returns the top 10 students with the highest CGPA.

Script

```
students = LOAD '/student_depression/student_lifestyle_100k.csv'
    USING PigStorage(',')
    AS (
        Student_ID:chararray,
        Age:chararray,
        Gender:chararray,
        Department:chararray,
        CGPA:chararray,
        Sleep_Duration:chararray,
        Study_Hours:chararray,
        Social_Media_Hours:chararray,
        Physical_Activity:chararray,
        Stress_Level:chararray,
        Depression:chararray
    );

data = FILTER students BY Student_ID != 'Student_ID';
typed = FOREACH data GENERATE
    Student_ID,
    Age,
    Gender,
    Department,
    (double)CGPA AS CGPA,
    Sleep_Duration,
```

```

Study_Hours,
Social_Media_Hours,
Physical_Activity,
Stress_Level,
Depression;

sorted = ORDER typed BY CGPA DESC;
top_10 = LIMIT sorted 10;
STORE top_10
    INTO '/student_depression/pig1_output'
    USING PigStorage('|');

```

Execution

```

ing aggregation.
2026-09-10 21:58:59,430 [main] INFO org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MapReduceLauncher - Success!
2026-09-10 21:58:59,446 [main] INFO org.apache.pig.Main - Pig script completed in 2 minutes, 7 seconds and 747 milliseconds (127747 ms)

```

Output

```

7549|20|Male|Business|4.0|8.6|7.6|2.4|29|6|False
52867|19|Female|Arts|4.0|8.2|7.1|4.9|90|2|False
45812|24|Female|Arts|4.0|8.5|7.6|1.4|6|6|False
31706|24|Male|Arts|4.0|6.4|7.9|4.4|83|5|False
22545|22|Male|Business|4.0|4.3|9.3|3.6|126|6|False
62938|19|Male|Engineering|4.0|7.2|7.4|4.2|29|3|False
12376|18|Male|Arts|4.0|6.6|9.0|4.8|79|2|False
62941|20|Male|Medical|4.0|6.9|7.4|1.8|11|4|True
23841|22|Female|Medical|4.0|9.2|8.1|4.2|110|4|False
12942|21|Male|Business|4.0|8.8|7.3|3.2|77|5|False

```

2. Average Study Hours by Department: Returns the average study hours of students for each academic department.

Script

```

students = LOAD '/student_depression/student_lifestyle_100k.csv'
    USING PigStorage(',')
    AS (
        Student_ID:chararray,
        Age:chararray,
        Gender:chararray,

```

```

        Department:chararray,
        CGPA:chararray,
        Sleep_Duration:chararray,
        Study_Hours:chararray,
        Social_Media_Hours:chararray,
        Physical_Activity:chararray,
        Stress_Level:chararray,
        Depression:chararray
    );
data = FILTER students BY Student_ID != 'Student_ID';
typed = FOREACH data GENERATE
    Department,
    (double)Study_Hours AS Study_Hours;
grouped = GROUP typed BY Department;
result = FOREACH grouped GENERATE
    group AS Department,
    AVG(typed.Study_Hours) AS Average_Study_Hours;
STORE result
    INTO '/student_depression/pig2_output'
    USING PigStorage('|');

```

Execution

```

2026-09-10 22:09:04,658 [main] INFO org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MapReduceLauncher - Success!
2026-09-10 22:09:04,677 [main] INFO org.apache.pig.Main - Pig script completed in 23 seconds and 422 milliseconds (23422 ms)

```

Output

```

Arts|4.504655465546545
Medical|4.501688038469237
Science|4.506875591649633
Business|4.553430436966337
Engineering|4.481208555616454

```

3. Average Sleep Duration by Depression Status: Returns the average sleep duration of students for each depression status

Script

```
students = LOAD '/student_depression/student_lifestyle_100k.csv'
    USING PigStorage(',')
    AS (
        Student_ID:chararray,
        Age:chararray,
        Gender:chararray,
        Department:chararray,
        CGPA:chararray,
        Sleep_Duration:chararray,
        Study_Hours:chararray,
        Social_Media_Hours:chararray,
        Physical_Activity:chararray,
        Stress_Level:chararray,
        Depression:chararray
    );
data = FILTER students BY Student_ID != 'Student_ID';
typed = FOREACH data GENERATE
    Depression,
    (double)Sleep_Duration AS Sleep_Duration;
grouped = GROUP typed BY Depression;
result = FOREACH grouped GENERATE
    group AS Depression,
    AVG(typed.Sleep_Duration) AS Average_Sleep_Duration;
STORE result
    INTO '/student_depression/pig3_output'
    USING PigStorage('|');
```

Execution

```
2026-09-10 22:11:52,510 [main] INFO org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MapReduceLauncher - Success!
2026-09-10 22:11:52,528 [main] INFO org.apache.pig.Main - Pig script completed in 22 seconds and 967 milliseconds (22967 ms)
```

Output

```
D:\Website\Varsity Notes  
True|6.738660306102127  
False|7.025262958927269
```

4. Average CGPA by Depression Status: Returns the average CGPA of students for each depression status.

Script

```
students = LOAD '/student_depression/student_lifestyle_100k.csv'  
    USING PigStorage(',')  
    AS (  
        Student_ID:chararray,  
        Age:chararray,  
        Gender:chararray,  
        Department:chararray,  
        CGPA:chararray,  
        Sleep_Duration:chararray,  
        Study_Hours:chararray,  
        Social_Media_Hours:chararray,  
        Physical_Activity:chararray,  
        Stress_Level:chararray,  
        Depression:chararray  
    );  
data = FILTER students BY Student_ID != 'Student_ID';  
typed = FOREACH data GENERATE  
    Depression,  
    (double)CGPA AS CGPA;  
grouped = GROUP typed BY Depression;  
result = FOREACH grouped GENERATE  
    group AS Depression,  
    AVG(typed.CGPA) AS Average_CGPA;  
STORE result
```

```
INTO '/student_depression/pig4_output'

USING PigStorage('|');
```

Execution

```
2026-09-10 22:15:14,401 [main] INFO org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MapReduceLauncher - Success!
2026-09-10 22:15:14,417 [main] INFO org.apache.pig.Main - Pig script completed in 24 seconds and 915 milliseconds (24915 ms)
```

Output

```
True|2.6135301132975592
False|2.9301766772665534
```

5. Top 10 Students By Study Hours: Returns the top 10 students with the highest study hours.

Script

```
students = LOAD '/student_depression/student_lifestyle_100k.csv'
    USING PigStorage(',')
    AS (
        Student_ID:chararray,
        Age:chararray,
        Gender:chararray,
        Department:chararray,
        CGPA:chararray,
        Sleep_Duration:chararray,
        Study_Hours:chararray,
        Social_Media_Hours:chararray,
        Physical_Activity:chararray,
        Stress_Level:chararray,
        Depression:chararray
    );

data = FILTER students BY Student_ID != 'Student_ID';
typed = FOREACH data GENERATE
    Student_ID,
    Age,
    Gender,
```



```

Department,
CGPA,
Sleep_Duration,
(double)Study_Hours AS Study_Hours,
Social_Media_Hours,
Physical_Activity,
Stress_Level,
Depression;

sorted = ORDER typed BY Study_Hours DESC;
top_10 = LIMIT sorted 10;
STORE top_10
    INTO '/student_depression/pig5_output'
    USING PigStorage('|');

```

Execution

```

2026-09-10 22:21:42,993 [main] INFO org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MapReduceLauncher - Success!
2026-09-10 22:21:43,014 [main] INFO org.apache.pig.Main - Pig script completed in 2 minutes, 8 seconds and 484 milliseconds (128484 ms)

```

Output

```

59878|23|Female|Science|3.24|7.8|12.8|3.0|21|4|False
21643|20|Male|Business|2.28|7.1|12.6|2.5|73|5|False
13260|24|Female|Engineering|3.66|7.5|12.3|3.5|92|4|False
81969|22|Male|Engineering|3.0|7.1|12.3|2.6|109|5|False
56999|23|Female|Business|3.83|8.8|12.3|6.1|44|5|False
9769|18|Male|Science|2.49|5.5|12.2|4.6|2|7|True
85668|21|Male|Medical|2.63|8.5|12.1|0.7|116|4|False
29095|23|Male|Engineering|2.31|9.8|12.0|4.6|10|8|False
81451|18|Male|Engineering|2.93|6.9|11.9|4.0|22|5|False
78826|24|Female|Science|3.55|7.8|11.9|2.5|63|4|False

```